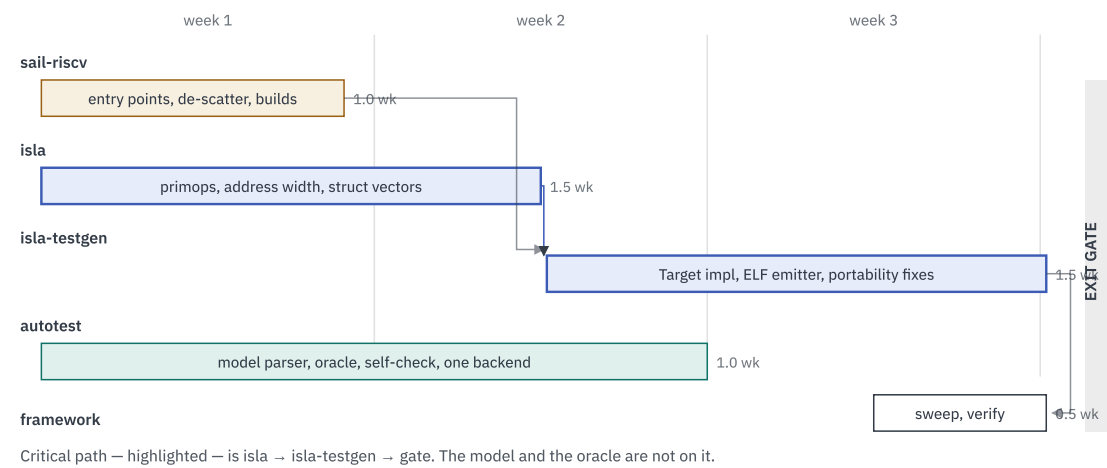


# Foundation, Repository by Repository

Everything needed before one instruction can be generated and pass on two simulators. The list is specific because each item has already been hit once — this is scoping, not estimating.

## Schedule



3 calendar weeks, 5.5 engineering-weeks. Three lanes start on day one and need nothing from each other.

## 1 · sail-riscv — the Golden Model

| Expose the entry points the generator needs |                                                                                                                                                                                                                                                                                                | 1.0 wk · week 1 · not on critical path |
|---------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------|
| TASK                                        | DETAIL                                                                                                                                                                                                                                                                                         | DAYS                                   |
| Two entry points                            | <code>isla_testgen_init</code> and <code>isla_testgen_step</code> in <code>main.sail</code> , modelled on the existing <code>isla_footprint</code> pair. <code>fetch()</code> made non-private so the second can call it                                                                       | 1                                      |
| Codegen workaround                          | Functions destructuring a $\geq 2$ -tuple parameter get inconsistent synthetic names in the IR. Destructure in the body instead — mechanical, ~50 functions across <code>core/</code> , <code>sys/</code> , <code>extensions/</code> , <code>pmp/</code>                                       | 1                                      |
| De-scatter 4 CSR accessors                  | <code>read_CSR</code> , <code>write_CSR</code> , <code>is_CSR_accessible</code> , <code>is_CSR_exception_virtual</code> hit the same limitation in the scattered-function <i>merging</i> step. ~302 clauses across ~17 files, consolidated one function each, first-match-wins order preserved | 2                                      |
| Concrete PMP reset                          | <code>reset_pmp()</code> builds a concrete entry per <code>pmpcfg</code> rather than updating the prior symbolic value                                                                                                                                                                         | 0.5                                    |
| Two builds                                  | Ordinary simulator, plus <code>-DCOVERAGE=ON</code> for the span manifest. Kept separate so a coverage run cannot use an uninstrumented emulator                                                                                                                                               | 0.5                                    |

The de-scatter is the largest single diff of the whole foundation and touches the most files. It is proposed upstream as its own commit so it can be reviewed on its merits, separately from the entry points. Nothing in it changes architectural behaviour.

2 · isla — the symbolic library

|                                        |                                                                                                                                                                                                                                                                                                                |                                    |
|----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------|
| Make the engine able to run this model |                                                                                                                                                                                                                                                                                                                | 1.5 wk · weeks 1-2 · critical path |
| TASK                                   | DETAIL                                                                                                                                                                                                                                                                                                         | DAYS                               |
| Track the Sail release                 | The Sail plugin consumes Sail's Jib IR, whose type definitions and rewrite-pass ordering change between releases. Update against the version pinned at project start                                                                                                                                           | 1.5                                |
| Missing primitives                     | The reservation set ( <code>load</code> , <code>match</code> , <code>valid</code> , <code>cancel</code> ), <code>count_trailing_zeros</code> with its <code>BV</code> support, <code>vector_init</code> , <code>sys_enable_experimental_extensions</code> . Each small; the risk is finding a further one late | 2                                  |
| Address width                          | The memory overlap check assumes 64-bit addresses. Take the width from the address's own solver-tracked value, so RV32 reads are not mis-sized                                                                                                                                                                 | 0.5                                |
| Struct-typed vector elements           | PMP configuration is <code>vector(64, Pmpcfg_ent)</code> — a vector of bitfield <i>structs</i> . The value encoding has no case for them, so any symbolic PMP access fails. Vector read and update build a per-field conditional instead                                                                       | 3                                  |

The last row is why PMP is reachable at all. Without it there is no symbolic PMP testing, and PMP is a required RFP item. It is the single highest-value fix in the foundation, and it is why this lane sits on the critical path.

3 · isla-testgen — the generator

|                                  |                                                                                                                                                                                                                             |                                                    |
|----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------|
| Teach the generator about RISC-V |                                                                                                                                                                                                                             | 1.5 wk · weeks 2-3 · critical path · needs 1 and 2 |
| TASK                             | DETAIL                                                                                                                                                                                                                      | DAYS                                               |
| Target implementation            | The generator's target interface — around thirty methods covering register naming, PC alignment, entry points, exception stop functions, ELF emission                                                                       | 3                                                  |
| ELF emitter                      | Preamble, initial register loads, the instruction under test, the final-state comparison, the trap vector, the HTIF <code>tohost</code> write                                                                               | 2.5                                                |
| Concrete boot PC                 | The engine sets the initial PC by asserting equality at the solver level, which does nothing when the PC is already concrete — as it is here, since <code>init</code> calls <code>reset()</code> . Write the value directly | 0.5                                                |
| PC width vs address width        | RV32's PC is 32 bits; the model's memory interface uses 64-bit addresses regardless of XLEN. Zero-extend for the fetch                                                                                                      | 0.5                                                |
| GPR numbering                    | Registers are assumed to start at zero. <code>x0</code> is hardwired and not general-purpose, so the interface gains a first-GPR index                                                                                      | 0.5                                                |
| IR and ISA config                | Build <code>riscv{32,64}.ir</code> with the preserve flags, and the <code>.toml</code> naming the toolchain, PC register and memory regions                                                                                 | 0.5                                                |

4 · autotest — the concrete oracle

|                                                     |                                                             |                               |
|-----------------------------------------------------|-------------------------------------------------------------|-------------------------------|
| The second path, which depends on none of the above |                                                             | 1.0 wk · weeks 1-2 · parallel |
| TASK                                                | DETAIL                                                      | DAYS                          |
| Config reader                                       | One Golden Model configuration becomes one validated object | 0.5                           |
|                                                     |                                                             |                               |

| TASK                   | DETAIL                                                                                                                                                  | DAYS |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|------|
| Model parser           | Parse <code>mapping clause assembly</code> and the mnemonic tables it references, so mnemonics, operand kinds and operand order all come from the model | 2    |
| Oracle                 | Build a probe that prints its result registers, run it on <code>sail_riscv_sim</code> , parse the values back                                           | 1    |
| Self-check scaffolding | Seed the output registers, wrap a body in a probe, bake the captured values into an asserting test                                                      | 1    |
| Toolchain layer        | Assemble and link exactly as the Golden Model does, reusing its <code>crt0.S</code> and linker script                                                   | 0.5  |

**This lane is the schedule's insurance.** It needs no solver, no IR and no upstream change, so it cannot be blocked by lanes 1–3. If the symbolic path slips, a working generator still exists.

## 5 · framework — orchestration

| Drive it, and prove it works |                                                                                                                                      | 0.5 wk · week 3 · needs a working generator |
|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------|
| TASK                         | DETAIL                                                                                                                               | DAYS                                        |
| Path resolution              | Every tool and directory resolved once, from environment then repo-relative then <code>PATH</code> , so a fresh clone needs no edits | 0.5                                         |
| Opcode sourcing              | Mnemonics and encodings from the model rather than an external opcode list                                                           | 1                                           |
| Single-file sweep            | Parse one Sail file, render a legal instance, assemble it for the true encoding, generate, then <i>run</i> the result                | 1                                           |

**The sweep runs the ELF, it does not just generate it.** Generation not erroring is not proof — a self-targeting `jal` “succeeds” at generation time and hangs forever when executed.

## Totals and the gate

| REPOSITORY   | WEEKS | STARTS                                                | BLOCKED BY           | ON CRITICAL PATH |
|--------------|-------|-------------------------------------------------------|----------------------|------------------|
| sail-riscv   | 1.0   | week 1                                                | nothing              | no               |
| isla         | 1.5   | week 1                                                | nothing              | yes              |
| isla-testgen | 1.5   | week 2                                                | isla, sail-riscv     | yes              |
| autotest     | 1.0   | week 1                                                | the model build only | no               |
| framework    | 0.5   | week 3                                                | either generator     | yes              |
| Total        | 5.5   | 3 calendar weeks with three lanes running in parallel |                      |                  |

EXIT GATE — END OF WEEK 3

`addi x1, x0, 1` generates from **both** backends, assembles with the real toolchain, and **passes** on `sail_riscv_sim` and on Spike.

Plus: the entry points survive an `isla-sail` rebuild, verified by a post-build check rather than assumed; and a symbolic PMP register read completes without crashing the engine, which is what proves the struct-typed vector fix landed.

**Why the gate is a passing test rather than a working pipeline.** Generation completing proves nothing — the failure mode this project keeps meeting is a test that runs and checks nothing. The gate is the narrowest artefact that could not be produced by a broken pipeline.

## What this de-risks

Every technical unknown in the engagement resolves in these three weeks, on one instruction, before any breadth is attempted:

| UNKNOWN                                  | RESOLVED BY                                                         |
|------------------------------------------|---------------------------------------------------------------------|
| Does the engine run this model at all?   | Lane 2 — primitives and the Sail release                            |
| Will upstream accept the entry points?   | Lane 1 — submitted in week 1, so review starts as early as possible |
| Can PMP be tested symbolically?          | Lane 2 — the struct-typed vector fix, checked at the gate           |
| Does the emitter produce a loadable ELF? | Lane 3 — run on two simulators, not just linked                     |
| Does the oracle path work independently? | Lane 4 — it never touches lanes 1–3                                 |
| Is a generated test capable of failing?  | The gate — a passing test on an independent simulator               |

**If something here is going to be impossible, it becomes visible in week three** rather than in week ten. That is the whole reason the foundation is built as one narrow slice instead of one layer at a time.

Task lists and file names reflect work already carried out once to de-risk this proposal; durations are estimates for repeating it against the model revision agreed at project start.